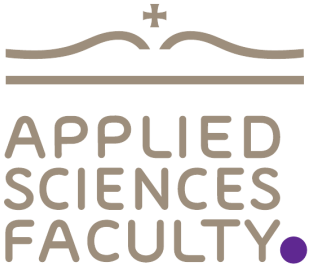
C++ — додаток. Qt. Lecture 2 — "Hello, world!"





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Qt "Hello, world!"

Почнемо із традиційної простенької програми:

```
#include <QApplication>
#include <QLabel>

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);
    QLabel label("Hello, world!");
    label.show();
    return app.exec();
}
```

Програмка ще зовсім скромна:

```
int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    QLabel label("Hello, world!");
    label.show();

    return app.exec();
}
```



але вже містить багато важливих елементів. Розберемо її.

- **QApplication** і **QLabel** — класи Qt, які представляють ті чи інші його елементи.
- Оголошення кожного класу знаходиться у файлі заголовків із тим же іменем, що і клас, тому на початку програми пише:

```
#include <QApplication>
#include <QLabel>
```

QApplication

- Один і тільки один об'єкт класу QApplication повинен бути в кожній програмі, яка використовує бібліотеку віджетів QtWidgets. Інакше слід використовувати QGuiApplication або QCoreApplication. Його роль — керування всіма глобальними ресурсами, здійснення всілякої необхідної прихованої магії.
- Його конструктору передаються аргументи командного рядка, з якими була викликана програма: `argc` — їх кількість та масив стрічок із їх вмістом — `argv[]`. Справа в тому, що є певні аргументи командного рядка, які здатне інтерпретувати саме Qt, тому слід дати йому шанс їх побачити.
- Цей клас ініціалізує дуже багато компонент програми, тому його слід створити **ДО** створення об'єктів будь-яких інших класів Qt.
- Детальніше про нього див. документацію, наприклад: `http://qt-project.org/doc/qt-5/qapplication.html#details`
- Будь-яка частина програми може отримати вказівник на її об'єкт класу QApplication за допомогою **статичного** методу **instance()**, оголошеного в одному із предків QApplication, класі QCoreApplication:

  QCoreApplication * QCoreApplication::instance()

  або скориставшись макросом qApp.

QLabel

- QLabel — клас для виведення статичних елементів, таких як текст або зображення. Текст може бути форматованим. Також може виводити відео.
- Не вміє взаємодіяти із користувачем.
- Найпростіше використання — конструктору передається текстова стрічка:

```
QLabel label("Hello, world!");
```

  Про інші способи використання — дещо пізніше.
- Документація: `http://qt-project.org/doc/qt-5/qlabel.html`

Щоб той чи інший візуальний об'єкт з'явився на екрані, слід "показати" його:

```
label.show();
```

Ця функція показує віджет, для якого викликана, разом із усіма нащадками у його вікні-предку. Якщо, як у нашому випадку, предка немає, вікно, на якому буде знаходитися віджет, створюється автоматично.

Сформувавши вигляд програми, запускаємо **цикл обробки повідомлень**:

```
return app.exec();
```

Коли він завершиться, програму теж можна завершувати.

Одна мітка — не дуже цікаво

Давайте додамо кнопку.

- Тепер в програмі буде два віджети — мітка і кнопка. Треба якось їх "прикріпити" один до одного.
- В більшості систем GUI для цього використовуються так звані layout-менеджери (менеджери розмітки?). Їхнє призначення — керувати взаємним розташуванням віджетів.
- Qt має великий комплект layout-менеджерів. Поміж них — QVBoxLayout, нащадок QLayout, який розташовує свої компоненти вертикально, одна під одною.
- Зверніть увагу також на його близького родича Q**H**BoxLayout, який розташовує віджети горизонтально.
- Повний список див. тут: `http://qt-project.org/doc/qt-5/layout.html`

Володіння вказівниками

Увага!

- Віджети, якими керуватимуть інші віджети (такі як layout-менеджери), слід виділяти динамічно!
- Це вимагається для спрощення питань володіння та відповідальності – хто має звільняти пам'ять і коли.
- Звільнятимуть їх пам'ять вже "нові власники".
- Тобто, віджет виділяється динамічно, за допомогою new, вказівник передається власнику, (наприклад, менеджеру розташувань), після чого його видаляти не потрібно — цим займеться деструктор власника.

Всі

< необхідні інклуди −−− по одному для кожного класу >

```
int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    QLabel *label = new QLabel("Hello, world!");
    QPushButton *quitButton = new QPushButton("&Quit");

    QVBoxLayout *layout = new QVBoxLayout;
    layout->addWidget(label);
    layout->addWidget(quitButton);

    QWidget window;
    window.setLayout(layout);

    window.show();

    return app.exec();
}
```

```
int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

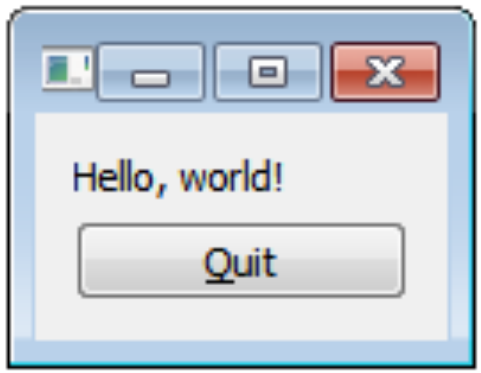
    QLabel *label = new QLabel("Hello, world!");
    QPushButton *quitButton = new QPushButton("&Quit");

    QVBoxLayout *layout = new QVBoxLayout;
    layout->addWidget(label);
    layout->addWidget(quitButton);

    QWidget window;
    window.setLayout(layout);

    window.show();

    return app.exec();
}
```



Проаналізуємо цю програму детальніше.

- Створюємо мітку з надписом та кнопку з текстом Quit:

```
QLabel *label = new QLabel("Hello, world!");
QPushButton *quitButton = new QPushButton("&Quit");
```

- Символ & в "&Quit" означає, що наступна за ним літера буде "shortcat-ом", клавіатурною комбінацією — натиснути кнопку можна буде не тільки мишкою, але й комбінацією Alt-Q.
- Створюємо layout-менеджер.

```
QVBoxLayout *layout = new QVBoxLayout;
```

- Доручаємо йому керувати нашими віджетами.

```
layout->addWidget(label);
layout->addWidget(quitButton);
```

- Хоча видимі віджети вміють відобразити самі себе у власному вікні (за допомогою методу show()), layout так не вміє.
- Тому потрібно створити вікно для нього:

```
QWidget window;
window.setLayout(layout);
```

- Вікно не в динамічній пам'яті, щоб не треба було його явно звільняти. Проблем із володінням, описаних раніше, при цьому не виникне — воно є головним вікном програми, тому не належить іншим вікнам.
- Нарешті показуємо вікно із нашими віджетами:

```
window.show();
```

- і запускаємо цикли обробки повідомлень:

```
return app.exec();
```

Сигнали і слоти

При всій симпатичності, наша кнопка Quit не функціюватиме.

- Програма не знає, як її реагувати на цю кнопку. Їй слід пояснити.
- Для цього використовується механізм сигналів та слотів.
- **Сигнал** — певна сутність, яка генерується, коли відбулася якась подія. Приклад такої події – натискання кнопки.
- Слот – функція, яка буде викликана у відповідь на певний сигнал.
- Сигнали не знають, чи до них хтось приєднаний, як і слоти не знають, чи вони до когось приєднані. Слот може бути приєднаним до декількох сигналів і навпаки. Сигнал навіть може бути приєднаним до іншого сигналу, зразу генеруючи і його.
- Будь які нащадки класу **QObject** можуть мати свої слоти та сигнали. Детальніше див. `http://qt-project.org/doc/qt-5/signalsandslots.html`.

Реакція на кнопку

Отож, повідомляємо Qt про призначення кнопки:

```
QObject::connect(quitButton, SIGNAL(clicked()),
                 qApp, SLOT(quit()));
```

Тобто, у відповідь на генерацію сигналу `clicked()`, кнопкою quitButton, слід виконати слот `quit()` поточного об'єкту QApplication, доступ до якого отримуємо за допомогою вказівника qApp.

Параметри цієї функції наступні:

- `quitButton` — об'єкт, чий сигнал приєднуємо.
- `SIGNAL(clicked())` — який саме сигнал.
- `qApp` — до слота якого об'єкта приєднуємо.
- `SLOT(quit())` — який саме його метод служить слотом.

`quit()` — метод QApplication[1], який завершує роботу програми.

[1] Оголошений він в QCoreApplication: `void QCoreApplication::quit ()`

Продовжуємо розвивати нашу програму.

- Багато програм, перш ніж завершити роботу, перепитуються, чи справді користувач цього хоче.
- Зрозуміло, що запитуватися слід в слоті, приєднаному до сигналу. Тому слід створити власний слот — метод якогось класу.
- Типовим, часто — найзручнішим, способом є створити свій віджет, який міститиме в собі всі компоненти вікна. (В нашому прикладі — мітку та кнопку.)
- Природним предком класу нашого віджету буде **QWidget**.

- Створюючи віджет, слід робити все за правилами — оголошення відповідного класу помістити в файл заголовків, реалізацію у відповідний .cpp файл.
- Взагалі, робити так слід завжди — це загальноприйнята хороша практика. Однак, чому в нашому випадку робити так **необхідно** — зараз стане зрозуміло.

MainWidget.h

```
#ifndef MAINWIDGET_H
#define MAINWIDGET_H

#include Всі< необхідні – QLabel, QPushButton, QVBoxLayout, QWidget>

class MainWidget : public QWidget
{
    Q_OBJECT

public:
    MainWidget();

private slots:
    void quit();

private:
    QLabel *label;
    QPushButton *quitButton;
};

#endif // MAINWIDGET_H
```

moc — Meta-Object Compiler

Секрет необхідності створювати окремий файл заголовків (окрім того, що так варто робити завжди!) криється у одній критично важливій для Qt утиліті, Meta-Object Compiler – moc.

- Qt не покладається повністю на C++, вважаючи, що йому потрібні деякі додаткові засоби, в C++ не присутні.
- Щоб їх реалізувати, використовується moc — своєрідний препроцесор. Він реалізовує сигнали-слоти, свою власну систему RTTI[2] та динамічні властивості (dynamical properties).
- Базується вся та система мета-об'єктів на трьох "китах":
  - класі **QObject**, який є обов'язковим предком для всіх класів, що хочуть скористатися засобами, наданими moc,
  - макросі **Q_OBJECT**, який має бути використаним у закритій частині класу, та включає необхідну для підтримки всіх цих засобів машинерію,
  - і нарешті, власне метакомпілятор, moc.
- moc, зустрівши в оголошенні класу-нащадка QObject макрос Q_OBJECT, створює окремий cpp-файл, який містить весь необхідний код.
- Цей файл слід скомпілювати та компонувати разом із рештою коду програми.

[2] Run-time type information.

- По замовчуванню, Qt Creator викликає moc для файлів заголовків – "хідерів", тому варто, створюючи свій віджет, використати стандартний підхід C/C++, створивши .h файл для оголошень та .cpp файл для визначень частин віджету.
- Детальніше див. тут: `http://qt-project.org/doc/qt-5/metaobjects.html`.

Перш ніж перейти до реалізації, розглянемо цей файл.

- Першим в оголошенні віджету йде макрос Q_OBJECT:

```
class MainWidget : public QWidget
{
    Q_OBJECT
```

- Наступний код оголошує конструктор:

```
public:
    MainWidget();
```

Саме в його тілі (реалізації) наш віджет і створюватиметься.

- Наступні рядки дещо дивні:

```
private slots:
    void quit();
```

Увага! "`private slots`" — не є звичайним кодом C++! Це директива для moc. Вона стверджує, що далі йтимуть сигнали. В нашому випадку сигнал один — quit.

- Нарешті, приватні члени — вказівники на елементи віджету:

```
private:
    QLabel *label;
    QPushButton *quitButton;
```

Реалізація — MainWidget.cpp

Створення об'єкту здійснюється в конструкторі:

```
MainWidget::MainWidget(){
    label = new QLabel("Hello, world!");
    quitButton = new QPushButton("&Quit");
    QVBoxLayout *layout = new QVBoxLayout;
    layout->addWidget(label); layout->addWidget(quitButton);

    QObject::connect(quitButton, SIGNAL(clicked()), this, SLOT(quit()));

    setLayout( layout );

    setWindowTitle("HW application main window");
}
```

Зауважте:

- Замість qApp, connect тепер приєднує сигнал до нашого віджету, тому третім аргументом використано this.
- Використовується власний setLayout() нашого класу, успадкований від QWidget.
- Використано метод `setWindowTitle`, який встановлює заголовок вікна.

Заради створення власних слотів все й затіялося, тому ізюминкою є quit():

```
void MainWidget::quit()
{
    QMessageBox quitMsg;
    quitMsg.setWindowTitle("Hello, world!");
    quitMsg.setText("Do you really want to quit?");
    quitMsg.setStandardButtons(QMessageBox::Yes |
                               QMessageBox::Cancel);
    quitMsg.setDefaultButton(QMessageBox::Cancel);
    if (quitMsg.exec() == QMessageBox::Yes)
        QApplication::instance()->quit();
}
```

Як видно, Це звичайна функція-метод. Щоб розуміти, що вона робить, розглянемо клас QMessageBox.

QMessageBox — 1

- **QMessageBox** — клас для створення модальних[3] діалогів. Він питається щось в користувача і повертає його відповідь програмі.
- **setWindowTitle** — встановлює заголовок вікна діалогу.
- **setText** — встановлює текст повідомлення. На підставі цього повідомлення користувач і приймає рішення. Додаткові пояснення можна передати за допомогою методів setInformativeText та setDetailedText.
- **setStandardButtons** вибирає, які ж кнопки будуть присутні. Комбінуються різні кнопки за допомогою **побітового** OR, |.
- Повний список міститься в переліченні, enum QMessageBox::StandardButtons: Ok, Open, Save, Cancel, Close, Discard, Apply, Reset, RestoreDefaults, Help, SaveAll, Yes, YesToAll, No, NoToAll, Abort, Retry, Ignore, NoButton.

[3] Тобто, таких, що блокують роботу програми на час свого виконання.

QMessageBox — 2

- **setDefaultButton** вказує, яка клавіша буде активною по замовчуванню.
- Викликається діалог методом **exec()**. Цей метод повертає код натиснутої клавіші, тому можна зробити так:

```
if (quitMsg.exec() == QMessageBox::Yes)
    QApplication::instance()->quit();
```

За потреби цей діалог можна зробити не-модальним. Див., наприклад, відповідь тут: `http://stackoverflow.com/questions/3211490/how-to-make-unblocking-qmessagebox`.

QMessageBox — 3

- Для виведення простих діалогів можна скористатися також так званим статичним API, не створюючи окремі об'єкти:

```
if (
    QMessageBox::question(this, "Hello, world!",
            "Do you really want to quit?",
            QMessageBox::Yes | QMessageBox::Cancel,
            QMessageBox::Cancel) == QMessageBox::Yes
    )
{
        QApplication::instance()->quit();
}
```

  хоча такий підхід і є менш гнучким.
- Статична функція QMessageBox::question(), фактично, робить те, що описано вище – створює та налаштовує об'єкт діалогу, запускає його та повертає результат роботи.
- Детальніше див. `http://qt-project.org/doc/qt-5/qmessagebox.html`.

Головна програма, "int main()":

```
int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    MainWidget window;

    window.show();

    return app.exec();
}
```

особливих запитань викликати не повинна.


idget window;
w.show();
n app.exec();
Hello, world!
Quit
16
Hello, world!
Do you really want to quit?
Yes
Cancel